Data Mining Techniques
Assignment 2

Individual or Group Project (2 People) (note: *for a group project, the same team as in Assignment 1 must be used*)

DEADLINE: Sunday, June 9, 2024, 23:55

In this assignment you will use the same data as in Assignment 1.

Question 1: Study over time (40%)



In this question, Hugging Face will be used to annotate sentences from the dataset you have been given. Through Hugging Face you will have easy access to pre-built models for sentence annotation, such as BERT, GPT, RoBERTa and many others. These models have been trained on large datasets and have evolved in sentence annotation, providing high performance. For the purposes of this assignment we will use the comments column from the reviews file. If this has not already been done, the column will need preprocessing: we remove punctuation, convert all characters to lowercase, and remove symbols such as hashtags, emoticons, emojis, links and stopwords from the dataset. You may use any HuggingFace model for sentiment analysis. Examples include *sentiment-roberta-large-english, distilbert-base-multilingual-cased-sentiments-student, BERT-Sentiment-Classifier*, etc.

- Using HuggingFace, annotate (annotation process) as many 2019 comments as you can by sentiment (positive/negative/neutral) – *hint: you can split the dataset into smaller pieces (chunks) and annotate them in batches*. The final result must also include an identifier (id), i.e. you should end up with a csv file (or a dataframe) with 3 columns (id, review, sentiment).
- Do the same for the 2023 reviews.
- Compare the overall sentiment over time (e.g. a histogram for each year showing the distribution of positive/negative/neutral).
- Compare the sentiment per neighbourhood over time (***BONUS - 5%***).

Question 2: Sentiment Analysis (45%)

In this question you will need to create two new datasets from the data you annotated in the previous question. More specifically, you will need:

- A train.tsv file (80% of all data points) containing the data you will use to train your models. The training data carry the label positive, negative or neutral.

- A test.tsv file (20% of the data points) containing the data you will use to test your model and make predictions. For each review in the test set, your model must decide whether it expresses positive, negative or neutral sentiment.

Follow the instructions we presented in the tutorial and prepare the features for each review using:

- Tf-idf
- Word embeddings

Use Python's pickle library to save the features in *.pkl files. This way the features do not have to be recomputed from scratch every time you run your program; you can simply load them into memory with the corresponding load method.

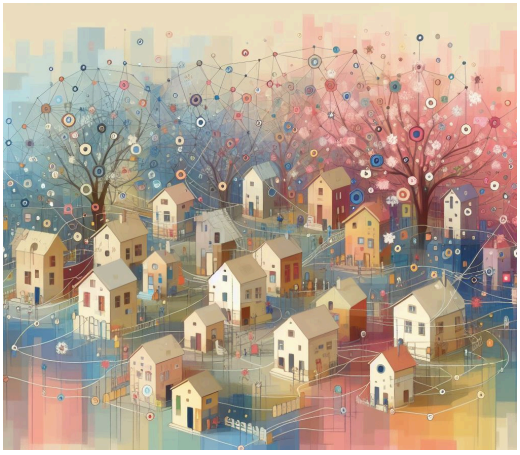
We test the following classifiers (testing will be done on the test data we have held out – it goes without saying that the test data must not be used for training!)

- SVM
- Random Forests
- KNN

Test your classifiers with both the TF-IDF features and the word embeddings. You must also evaluate and record the performance of each method using 10-fold Cross Validation with the following metrics:

- Precision / Recall / F-Measure
- Accuracy

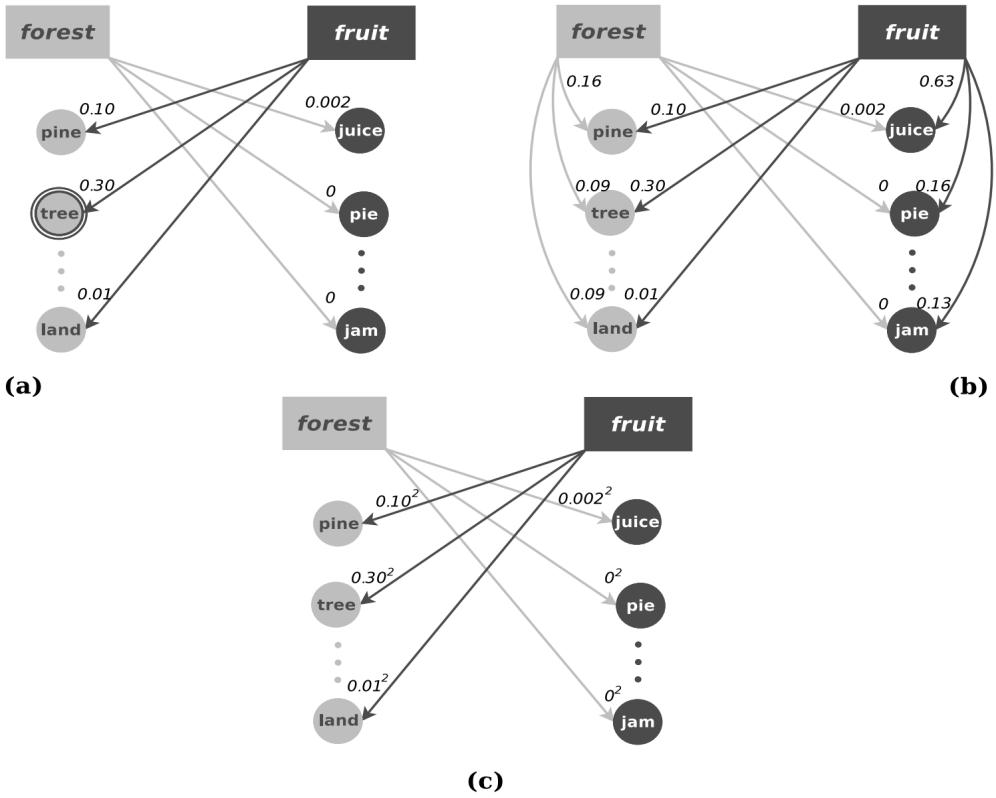
Question 3: Similarity (jaccard, cosine, etc) and semantic neighbourhoods. (15%)



You Shall Know a Word by the Company It Keeps

Using the comments column and word embeddings, create a function that computes the similarity between the vectors of two words (e.g. cosine or any other method you like; for examples, see the methods at this link). You may use any method that produces word embeddings (word2vec, fastText, GloVe, etc.) or use ready-made pre-trained embeddings.

Then, for two words (arbitrary words given by a user) and a parameter N, your code should compute (a) the semantic neighbourhood of the two words and (b) the similarity of the two words derived from their neighbourhoods according to the three figures below, i.e. 3 different similarities.



Pictorial view of neighborhood-based metrics. Two reference nouns, "forest" and "fruit",are depicted along with their neighborhoods: {pine, tree, . . . , land} and {juice, pie, . .

. , jam}, respectively. Arcs represent the similarities between reference nouns and neighbors. The similarity between "forest" and "fruit" is computed according to (a) maximum similarity of neighborhoods, (b) correlation of neighborhood similarities, and (c) sum of squared neighborhood similarities.
https://slp-ntua.github.io/potam/preprints/journal/2013_NLE_senetwork_DSM_draft.pdf

$$M_n(w_i, w_j) = \max\{\alpha_{ij}, \alpha_{ji}\},$$

$$\alpha_{ij} = \max_{x \in N_j} S(w_i, x), \quad \alpha_{ji} = \max_{y \in N_i} S(w_j, y).$$

(a) maximum similarity of neighborhoods

$$R_n(w_i, w_j) = \max\{\beta_{ij}, \beta_{ji}\},$$

$$\beta_{ij} = \rho(C_i^{N_i}, C_j^{N_i}), \quad \beta_{ji} = \rho(C_i^{N_j}, C_j^{N_j})$$

$$C_i^{N_i} = (S(w_i, x_1), S(w_i, x_2), \ldots, S(w_i, x_n)), \quad \text{where } N_i = \{x_1, x_2, \ldots, x_n\}$$

(b) correlation of neighborhood similarities

In the example of the figure, the similarities of the word forest (C1) with the neighbourhood (N1) are

$$C_1^{N_1} = (0.16, 0.09, \ldots, 0.09)$$

In the example of the figure, the similarities of the word fruit (C2) with the neighbourhood (N1) are

$$C_2^{N_1} = (0.10, 0.30, \ldots, 0.01)$$

In the same way, we also compute $C_2^{N_1}$ and $C_1^{N_2}$. Here *ρ* is the Pearson correlation coefficient.
https://stackabuse.com/calculating-pearson-correlation-coefficient-in-python-with-numpy/

$$E_n^{\theta}(w_i, w_j) = \left(\sum_{x \in N_j} S^{\theta}(w_i, x) + \sum_{y \in N_i} S^{\theta}(w_j, y) \right)^{\frac{1}{\theta}}$$

$$E_n^{\theta=2}(\text{“}forest\text{”}, \text{“}fruit\text{”}) = \sqrt{(0.10^2 + 0.30^2 + \cdots + 0.01^2) + (0.002^2 + 0^2 + \cdots + 0^2)}$$

(c) sum of squared neighborhood similarities

- How much does each similarity increase or decrease if we change the parameter N (the size of the neighbourhood)?
- Also, present some word neighbourhoods in any way you like, for an N of your choice.

Deliverable:

The assignment may be carried out by the same teams you formed in Assignment 1. If you worked individually, continue individually.

You will upload to eclass a folder named **sdixxxx_sdixxxx.zip** (where sdi are the student IDs of the team members; for an individual assignment a single sdi in the zip file name is enough). The zip file **MUST** contain a **Python notebook** through which someone can follow your work step by step (i.e. you will not submit the training/test data again).

The notebook also serves as the complete report for your assignment (you will not submit anything as doc or pdf), so design it carefully and remember to write a description at each step of what your code does in each cell. The notebook must be submitted “run”, with the results visible.

Also, for all questions (as far as feasible/useful), qualitative comments are essential regarding what you observe, what conclusions follow from the plots, etc. Clarifications about the assignment will be given via e-class.